

Experiment 3 | Lab Report

Pulse Width Modulation Using GPIO

Controlling the brightness of eight LEDs with PWM signals from a Raspberry Pi

Submitted by	Chavan Atharva Sunil
Roll Number	240003021
Date of Experiment	10 September 2026
Board Used	Raspberry Pi (BOARD numbering)

1 Aim

To control the **brightness** of eight LEDs connected to the GPIO pins of a **Raspberry Pi** using **pulse width modulation (PWM)**. Each pin is switched rapidly between HIGH and LOW, and the fraction of time it spends HIGH — the *duty cycle* — sets how bright its LED appears. The duty cycles are first held at two fixed levels to show that the brightness follows the duty cycle, and are then varied continuously in software to sweep a travelling wave of brightness along the eight LEDs.

Objectives

- To configure eight GPIO pins of the Raspberry Pi as PWM outputs using the RPi.GPIO library and wire one LED to each.
- To demonstrate that the perceived brightness of an LED is set by the duty cycle of the PWM signal driving it, by running two groups of LEDs at two different fixed duty cycles.
- To vary the duty cycle of each channel continuously in software so that every LED fades up and down, with the eight LEDs staggered in time to produce a wave of brightness across the array.
- To verify the programmed timing of the pattern against the recorded video.

Brief Theory

A digital GPIO pin has only two output states, HIGH (3.3 V) and LOW (0 V). It cannot produce an intermediate voltage directly, so an LED connected to it is, at any instant, either fully ON or fully OFF. Pulse width modulation gets around this by switching the pin ON and OFF repeatedly at a fixed frequency $f = 1/T$ and controlling only the fraction of each period for which it is HIGH. That fraction is the duty cycle,

$$D = \frac{t_{\text{on}}}{T} \times 100\%, \quad V_{\text{avg}} = D \cdot V_{\text{HIGH}}.$$

If the switching is fast enough — above about 100 Hz, well beyond the flicker fusion rate of the eye — the LED appears to glow steadily at a brightness proportional to the *average* current, which in turn is proportional to the duty cycle. A duty cycle of 100 % gives full brightness, 50 % roughly half, and 0 % is OFF. The switching frequency itself does not change the brightness; only the duty cycle does.

On the Raspberry Pi, RPi.GPIO provides PWM on any GPIO pin in software: a GPIO.PWM(pin, frequency) object is created for the pin, started with start(duty), and its duty cycle can be changed at any time with ChangeDutyCycle(duty) in the range 0–100. The library toggles the pin from a background thread, so eight independent PWM channels can run at once — which is what this experiment needs, since the Pi has only two hardware PWM channels.

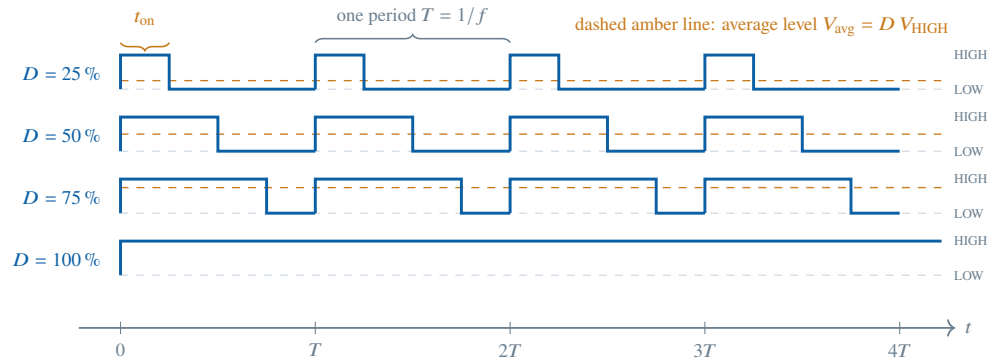


Figure 1: PWM waveforms at four duty cycles. The period T is the same in every row; only the HIGH fraction changes, and with it the average voltage seen by the LED.

Note on current limiting. As in the earlier experiments, the LEDs were connected *directly* to the GPIO pins, without series resistors, and the demonstration ran without damage. PWM does not change the peak current — during each ON interval the pin sources the same current as a steadily HIGH pin — so the usual advice still applies: a series resistor of roughly $220\ \Omega$ per LED, giving $(3.3 - 2)/220 \approx 6\ \text{mA}$, is the recommended practice for sustained use.

2 Materials Required

#	Component	Qty.	Purpose in the setup
1	Raspberry Pi (with SD card, OS installed)	1	Host board; generates the eight PWM signals
2	Solderless breadboard	1	Mounting the LEDs and the ground rail
3	LEDs (5 mm, red)	8	One per PWM channel
4	Male–female jumper wires	9–12	GPIO header to breadboard connections
5	USB power supply	1	Powering the Raspberry Pi
6	Monitor, keyboard and mouse	1	Running the programs from the terminal

3 Methodology

Eight LEDs were mounted on the breadboard in two columns of four, each anode wired directly to its own GPIO pin of the Raspberry Pi and all cathodes tied to a common ground rail returned to a GND pin of the board. The eight pins were configured as outputs and a software PWM object was created on each, all at the same switching frequency and all started at 0% duty cycle so that the display begins dark.

The experiment was then run in two parts. In **Part A** the duty cycles were held constant: four LEDs at 100% and the other four at 10%, giving a direct side-by-side comparison of a bright group and a dim group driven by the same kind of signal. In **Part B** the duty cycle of every channel was recomputed twenty times a second so that each LED fades smoothly up to full brightness and back down, with the eight channels offset in time, producing a wave of brightness that travels along the array.

3.1 GPIO connections

LED	Raspberry Pi pin	Part A duty cycle
LED 1	Pin 40	100 % (bright group)
LED 2	Pin 38	100 %
LED 3	Pin 36	100 %
LED 4	Pin 35	100 %
LED 5	Pin 33	10 % (dim group)
LED 6	Pin 32	10 %
LED 7	Pin 31	10 %
LED 8	Pin 29	10 %
All cathodes	GND	—

Numbering used here

The programs use **BOARD** numbering (`GPIO.setmode(GPIO.BOARD)`), so the numbers in the table are the *physical* positions on the 40-pin header rather than the Broadcom channel numbers. The same eight pins and the same wiring were used as for the binary display of Experiment 2; only the software changed. All eight channels share one switching frequency of 200 Hz, so the only thing that differs between LEDs is the duty cycle.

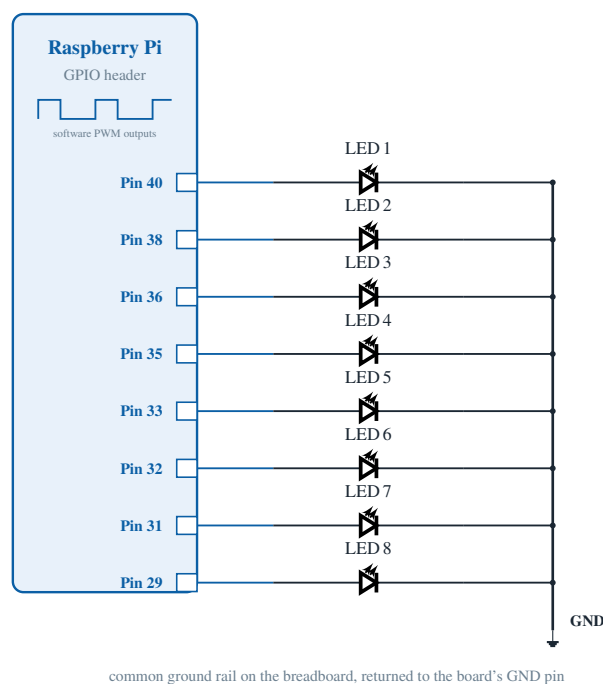


Figure 2: Circuit schematic. Eight LEDs are driven directly from eight GPIO pins, each carrying its own software-generated PWM signal, with all cathodes returned to the common ground rail on the breadboard.

3.2 Part A — two fixed duty cycles

The first program simply sets and holds the duty cycles: the first four channels at 100 % and the last four at 10 %. Because every channel is switched at the same 200 Hz, any difference in brightness between the two groups can only come from the duty cycle. At 100 % the pin is HIGH continuously and the LED is at full brightness; at 10 % it is HIGH for only 0.5 ms of every 5 ms period, so the LED receives one-tenth of the average current and glows visibly but much more dimly.

3.3 Part B — a travelling wave of brightness

The second program recomputes every duty cycle on a fixed schedule. Each LED follows the same cycle of length 6.4 s: its duty cycle rises linearly from 0 to 100 % over 1.6 s, falls back to 0 over the next 1.6 s, and stays at 0 for the remaining 3.2 s. Consecutive LEDs start this cycle 0.8 s apart, so the fade runs along the array like a wave. Since each LED is lit for exactly half its cycle and the eight start times span one full cycle, exactly four LEDs are lit at any instant: the newest is brightening, the oldest is dimming, and every 0.8 s one LED switches off as the next switches on.

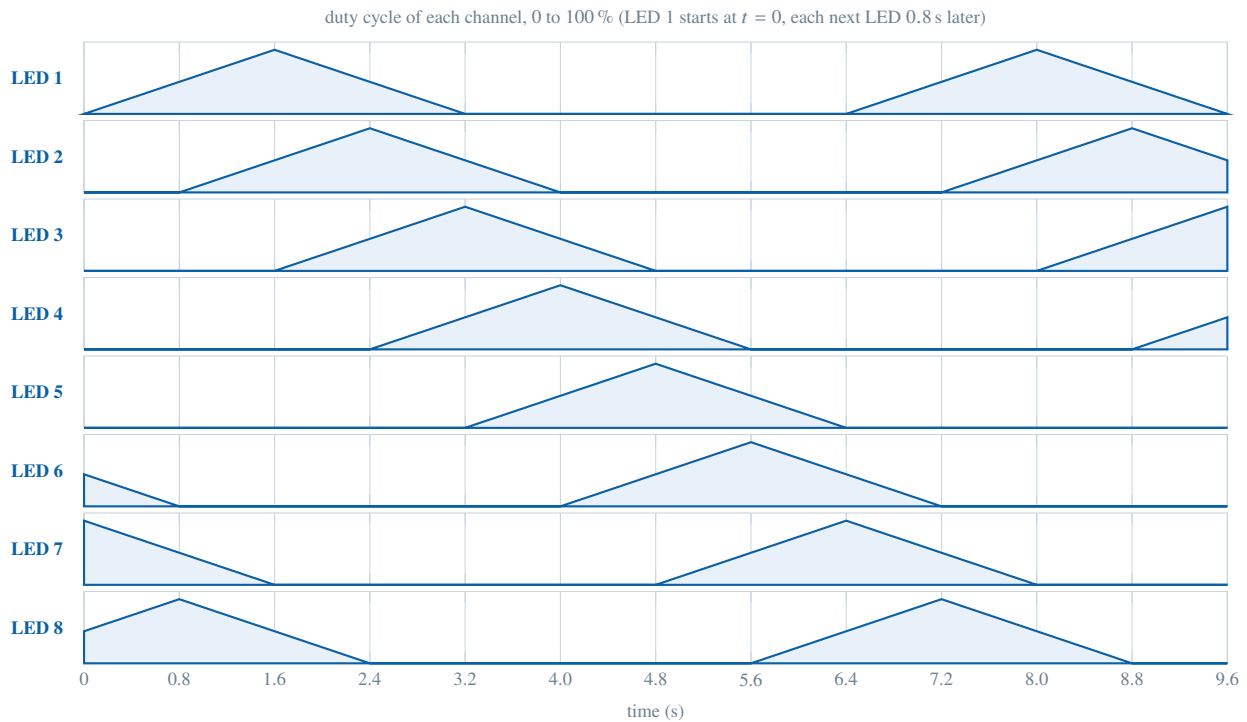


Figure 3: Programmed duty cycle of the eight channels in Part B. Each channel repeats a 6.4 s cycle — fade in, fade out, then dark — and neighbouring channels are staggered by 0.8 s, so four LEDs are lit at any moment and the pattern advances one LED every 0.8 s.

3.4 Procedure

1. Eight LEDs were placed on the breadboard in two columns of four, with all cathodes on the common ground rail, which was returned to a GND pin of the Raspberry Pi.
2. Each anode was wired to its own GPIO pin in the order given in the table above. The wiring was checked with the board powered off.
3. In the program, the eight pins were configured as outputs and a `GPIO.PWM` object was created for each at 200 Hz and started at 0% duty cycle.
4. **Part A:** the first four channels were set to 100 % and the last four to 10 % with `ChangeDutyCycle()`, and the two groups were compared side by side while the program held the values.
5. **Part B:** the second program was run. Every 20 ms it computed the position of each channel within its 6.4 s cycle, converted that to a duty cycle with the fade-in/fade-out profile of Figure 3, and applied it to the channel.
6. Both runs were recorded on video. The stills in this report are frames from those recordings, and the timing of the wave was checked by reading the brightness of each LED off the frames.
7. Finally, on `Ctrl+C`, every PWM channel was stopped with `stop()` and `GPIO.cleanup()` returned all the pins to inputs.

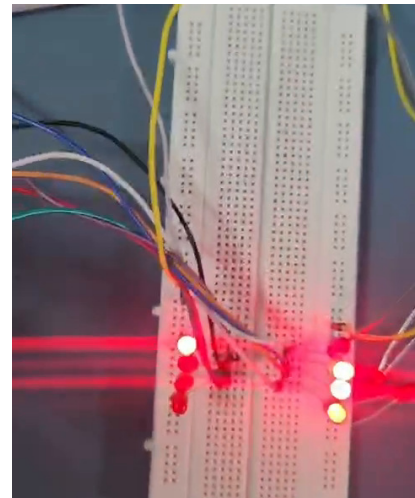


Figure 4: The assembled circuit — eight LEDs in two columns of four on the breadboard, each wired directly to its own GPIO pin through the bundle of jumper wires running to the Raspberry Pi's header (off the top of the frame), with the cathodes on the common ground rail.

3.5 Code (Python, Raspberry Pi)

Listing 1: Part A — two groups of LEDs held at two fixed duty cycles.

```

1 import RPi.GPIO as GPIO
2 import time
3
4 # Physical (BOARD) pin numbers of the eight LEDs, LED 1 first
5 LED_PINS = [40, 38, 36, 35, 33, 32, 31, 29]
6 PWM_FREQ = 200          # switching frequency in Hz, well above flicker
7
8 DUTY_BRIGHT = 100       # LEDs 1-4: full brightness
9 DUTY_DIM     = 10        # LEDs 5-8: one tenth of the average current
10
11 GPIO.setmode(GPIO.BOARD)
12 GPIO.setwarnings(False)
13
14 pwm = []
15 for pin in LED_PINS:
16     GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
17     channel = GPIO.PWM(pin, PWM_FREQ) # one software PWM channel per pin
18     channel.start(0)                  # begin with the LED off
19     pwm.append(channel)
20 try:
21     for channel in pwm[:4]:
22         channel.ChangeDutyCycle(DUTY_BRIGHT)
23     for channel in pwm[4:]:
24         channel.ChangeDutyCycle(DUTY_DIM)
25     print("LEDs 1-4 at", DUTY_BRIGHT, "% | LEDs 5-8 at", DUTY_DIM, "%")
26
27     while True:
28         time.sleep(1)          # hold the levels until Ctrl+C
29
30 except KeyboardInterrupt:
31     print("\nExiting program")
32
33 finally:
34     for channel in pwm:
35         channel.stop()
36     GPIO.cleanup()

```

Listing 2: Part B — a wave of brightness travelling along the eight LEDs.

```

1 import RPi.GPIO as GPIO
2 import time
3
4 LED_PINS = [40, 38, 36, 35, 33, 32, 31, 29]
5 PWM_FREQ = 200          # Hz
6 CYCLE = 6.4             # s, one fade-in / fade-out / dark cycle of an LED
7 STAGGER = CYCLE / 8     # 0.8 s between the start of neighbouring LEDs
8 TICK = 0.02             # s between duty-cycle updates
9
10 def duty_cycle(phase):
11     """Duty cycle (0-100 %) for a position in the cycle, phase in [0, 1)."""
12     if phase < 0.25:
13         return 100 * phase / 0.25
14     if phase < 0.5:
15         return 100 * (0.5 - phase) / 0.25
16     return 0             # dark for the second half
17
18 GPIO.setmode(GPIO.BOARD)
19 GPIO.setwarnings(False)
20 pwm = []
21 for pin in LED_PINS:
22     GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
23     channel = GPIO.PWM(pin, PWM_FREQ)
24     channel.start(0)
25     pwm.append(channel)
26 try:
27     t0 = time.time()
28     while True:
29         t = time.time() - t0
30         for i, channel in enumerate(pwm):
31             phase = ((t - i * STAGGER) / CYCLE) % 1.0
32             channel.ChangeDutyCycle(duty_cycle(phase))
33         time.sleep(TICK)
34
35 except KeyboardInterrupt:
36     print("\nExiting program")
37
38 finally:
39     for channel in pwm:
40         channel.stop()
41     GPIO.cleanup()

```

4 Result and Output

Both parts worked as intended. In **Part A** the four LEDs at 100 % duty cycle glowed at full brightness while the four at 10 % were clearly lit but much dimmer, with no other difference between the groups. In **Part B** the LEDs faded up and down in turn: **four were lit at any moment**, the pattern advanced by one LED every 0.8 s, and each LED was lit for 3.2 s and dark for 3.2 s — a 6.4 s cycle, exactly as programmed.

Observed outputs

Output	Run	Duty cycle set	What was observed
1	Part A, LEDs 1–4	100 % (held)	full brightness; the four LEDs of one column glow steadily with bright white centres in the video
2	Part A, LEDs 5–8	10 % (held)	clearly lit but much dimmer; the four LEDs of the other column show only a red glow, no bright centre
3	Part B, all eight	0 → 100 → 0 %, staggered	a wave of brightness travels along the LEDs; four lit at a time, advancing one LED every 0.8 s (measured from the video, Figure 7)

In Part A the switching frequency was identical for both groups, so the difference in brightness is due to the duty cycle alone. In Part B the timing read off the video — one LED switching on and one switching off every 0.8 s, each LED lit for 3.2 s — matches the constants $\text{STAGGER} = 0.8$ and $\text{CYCLE} / 2 = 3.2$ in Listing 2.

Part A — two fixed duty cycles

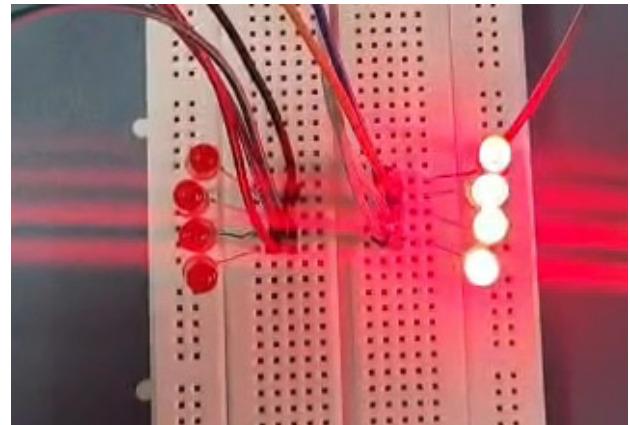
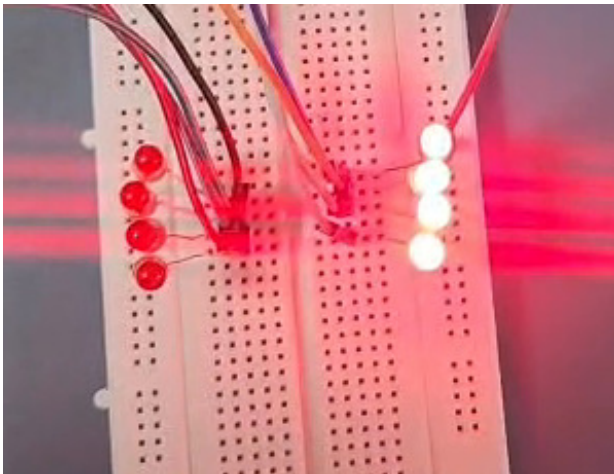


Figure 5: Part A, two frames from the recording. The right-hand column of four LEDs is driven at 100 % duty cycle and the left-hand column at 10 %. Both groups are lit, but the 10 % group is visibly dimmer — the camera shows only the red glow of the LED body, without the bright centre of the 100 % group.

Part B — the travelling wave

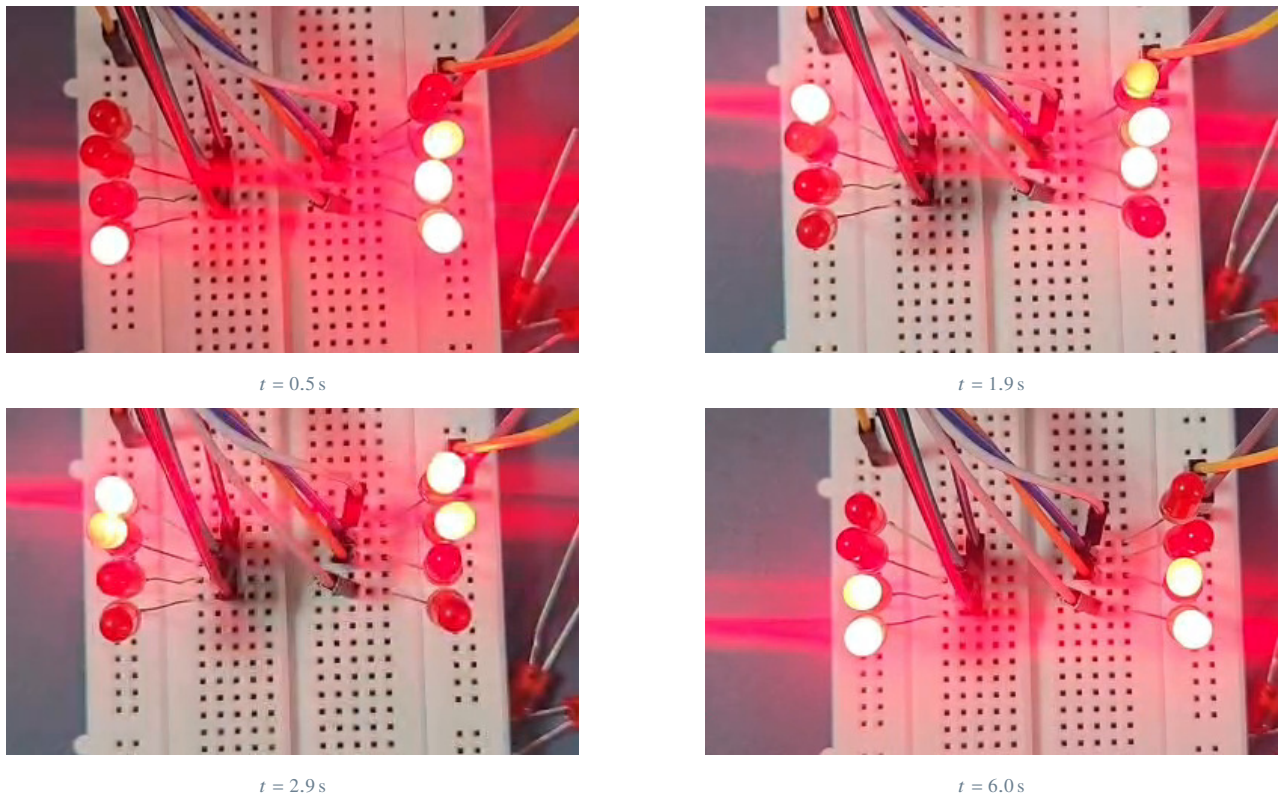


Figure 6: Part B at four instants of the same recording. In every frame four LEDs are lit. The LED that has just switched on is still brightening — it appears yellowish rather than white, since its duty cycle is still rising — while the LED about to switch off is dimming; the fully bright ones in between are at or near 100 %. Comparing the frames shows the lit group moving through the array.

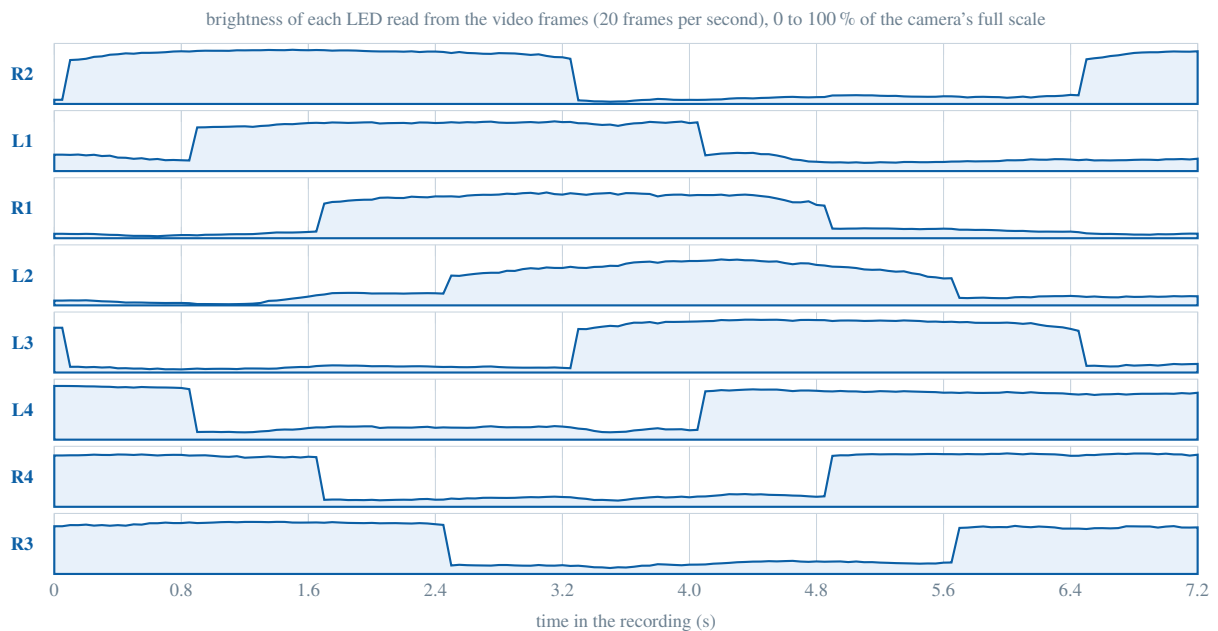


Figure 7: Part B, measured. The brightness of each LED was read from the video frames (the average green-channel value of the pixels covering the LED, which is the camera channel that saturates last) for the first 7.2 s of the recording. L and R denote the left and right columns of LEDs on the breadboard, numbered from the top; the panels are ordered by the time each LED switched on, which follows the program's pin order rather than the LEDs' physical positions. Each LED is lit for 3.2 s and dark for 3.2 s, and successive LEDs switch on 0.8 s apart — the same pattern as Figure 3. The camera saturates well below full brightness, which is why the fade-in and fade-out ramps of the programmed triangle appear as plateaus with short edges here.

Video demonstration and code

Google Drive link

The demonstration videos, this report and the complete source code are available in the following shared folder:

https://drive.google.com/drive/folders/11TP-NX_EF_IpxHIAUYmG8YWPQfst2ae2?usp=sharing

Access has been set to *anyone with the link can view*.

Discussion

- **Software PWM.** RPi.GPIO generates each PWM signal from a Python-level thread, so the timing is not as precise as hardware PWM and can jitter when the processor is busy. At 200 Hz with eight channels this was not visible to the eye, but it is why hardware PWM (available on only two channels of the Pi) is preferred where the exact frequency matters, for example for servo control.
- **The camera sees PWM differently from the eye.** In the Part A recording the bright group occasionally appears to dip for a frame or two at regular intervals. This is a beat between the 200 Hz switching and the camera's frame rate and rolling shutter, not a change in the programmed duty cycle: to the eye the brightness was steady. Likewise, the camera saturates at a duty cycle well below 100 %, which is why the smooth triangular fades of Part B come out as near-square pulses in Figure 7.
- **Perceived brightness is not linear in duty cycle.** The eye responds roughly logarithmically to light, so the step from 10 % to 100 % looks smaller than a factor of ten, and most of the visible change in a linear fade happens at the low end. A fade that looks uniform would use a duty cycle that rises exponentially (or as a square) rather than linearly.
- **Frequency and flicker.** At 200 Hz no flicker was visible at any duty cycle. At a much lower frequency (a few tens of hertz) the same program would show visible flicker at low duty cycles, and at 1–2 Hz the PWM signal becomes the blinking of Experiment 1 — PWM is simply that blinking made fast enough to blur into a steady brightness.
- **Current.** PWM reduces the *average* current but not the peak, so driving the LEDs without series resistors carries the same risk as before, and a 220 Ω resistor per LED remains the safer arrangement.

5 Conclusion

The experiment showed how a purely digital output can be made to behave like an adjustable one. By switching each GPIO pin rapidly and controlling only the fraction of time it spends HIGH, the Raspberry Pi set the brightness of eight LEDs independently: two fixed duty cycles produced two clearly different, steady brightness levels, and sweeping the duty cycles continuously in software produced a smooth wave of brightness travelling along the array, with four LEDs lit at a time and the pattern advancing every 0.8 s, exactly as programmed.

Where Experiment 1 used GPIO to switch LEDs and Experiment 2 used a group of pins as a parallel binary port, this experiment used time — the duty cycle — as the controlled quantity. The same technique extends directly to dimming lamps, driving motors at variable speed, and positioning servos, all of which are controlled with PWM from GPIO pins in the same way.